Response to Reviewer (Revision Round 3)

We thank the reviewer for a detailed and constructive report on the full manuscript. The recurring theme across review rounds has been that several statements were stronger than our design supports. We have taken that seriously and made a deliberate, paper-wide change of stance: **we claim less, and we claim it once.** The paper's spine is now *whole-stack idiomatic performance under a stated operating point*, with the same-SQL control presented as a **contrast that bounds** the raw-execution component rather than a decomposition that isolates mechanisms. We also ran the four targeted experiments the review pointed toward. All section and table references below are to the revised submission build (`ist/ist_main.tex`); a numbered online supplement (Supplement Tables S1–S20) accompanies it. Effective main-text length is 14,996 words, under the 15,000 limit (`word-count.pdf`).

Overarching change: claim less, decisively

Wherever the manuscript previously suggested that the same-SQL control *decomposes* the deficit or that "most of the cost lies upstream," it now states that the control **changes query generation, hydration, protocol, and mapping together as a bundle and isolates none of them individually**; it *bounds* the raw-execution component. This wording is now consistent across both abstracts, the Introduction, the Methodology estimand naming (renamed "same-SQL (raw-path) estimand ... a compound contrast, not a mechanism-isolating decomposition"), Results, Discussion, and Conclusion.

Major concerns

§6.1 — same-SQL framing overclaimed. Corrected as above. The estimand is now a compound contrast; Results and Discussion state explicitly that it does not attribute the deficit to any single component (Results §"Same-SQL control"; Methodology §"Analysis").

§6.2 — p99 was under-qualified. The primary p99 is now framed throughout as **saturated closed-loop tail latency at a 50-connection operating point against a ten-connection pool**, which includes connection-acquisition queueing (Abstract; Introduction; Methodology; Results §"Tail latency"). As the intrinsic-tail companion we added a **coordinated-omission-corrected open-loop** measurement, and in this round extended it to **both engines** (new experiment; Supplement Table S17): the sub-saturation corrected tail is flat and small and each layer collapses once the offered rate crosses its capacity on MySQL as on PostgreSQL, so the intrinsic-tail ordering is engine-robust.

§6.3 — "idiomatic" was underspecified. We added a consolidated `experiments/METHODOLOGY.md` and a compact **Supplement Table S16** documenting, per adapter: the exact eager-loading API used on the deep fetch, its round-trip count, the documented alternative loading strategy we did *not* use, and confirmation that query logging is disabled with no read-path hooks or validation (verified against source). We also ran an **alternative eager-loading sensitivity experiment** (Supplement Table S18): where the alternative is a byte-identical drop-in, switching the join-default data-mapper ORMs (Sequelize, MikroORM) to select-in makes them *slower* (0.68–0.75×), so their deficit is intrinsic rather than a poor default; conversely, Objection's batched-select-in default is 1.6× slower than a single join on MySQL, so part of *its* deficit there is a strategy choice. TypeORM's query strategy errors on the ordered nested `findOne` in the pinned version and is disclosed as a portability caveat rather than measured.

§6.4 — practical recommendations too broad. Recommendations are scoped to layer class and access pattern and bounded by the operating point; we removed unconditional phrasings.

§6.5 — Prisma mechanism conflated observation and inference. We now separate the **observation** (Prisma draws ~5× the application CPU at the fixed ten-connection pool; its ranking changes with concurrency) from the **documentation fact** (in-process multi-threaded query engine) from **inference**; the abstract CPU statement is qualified "at that operating point."

§6.6 — MySQL insert mechanism asserted without direct evidence. We ran the **performance_schema commit-flush experiment** (new; Supplement Table S19). Sampling the redo-log and binary-log flush wait instruments during the insert workload at default durability, the flush takes 0.57–0.69 ms per insert (38–40% of each insert) and is **nearly identical across the native driver, query builder, and slowest ORM** — a shared engine floor — and relaxing durability removes it and lifts throughput. The manuscript's "consistent with a shared MySQL-side commit cost" is now *directly measured* (Results §"RQ2"; Supplement §"MySQL insert commit-flush mechanism").

§6.7 — temporal concentration of measurements. We added a **post-restart robustness check** (new; Supplement Table S20): after restarting both database engines (cold buffer pools, fresh connections) we re-measured a representative deep-fetch subset eight days after the primary campaign. The layer ranking reproduces on both engines; absolute throughput is 3–23% lower, which is the day-to-day host drift that our *relative* analysis is designed to absorb. We disclose this is a single host, so it checks temporal robustness, not cross-machine generalization.

§6.8 — CPU/equal-CPU evidence belonged in the main text. We added a compact main-text **resource-footprint table** (Table 5: per-layer application CPU, database CPU, combined req/CPU-second, and peak RSS on the deep fetch), with the equal-CPU one-core headline in its caption; the fuller per-metric tables remain in the supplement. To stay within the length limit we moved four secondary tables (two read patterns, aggregation, and the same-SQL control) to the supplement.

Statistics (§8)

- **p99 inference beyond the deep fetch.** We extended paired inference to **all

five patterns × both engines** (per-cell bootstrap CIs and paired permutation/Wilcoxon on adjacent p99 pairs). Six of seven adjacent deep-fetch p99 pairs separate; five to six of seven separate across the reads on each engine; on MySQL inserts no adjacent p99 pair separates — the tail counterpart of the throughput engine floor (Results §"Tail latency").

- **Adjacent-rank tests.** These are defined by the *observed* ranking and were not

preregistered; we now call them **planned pairwise contrasts on the observed ranking, reported descriptively**, with Bonferroni retained only as a conservatism note.

- **Permutation resolution.** Corrected to $\mathbf{p = 1/20{,}001 \approx 5.0\times10^{-5}}$ (an equality

at the resolution floor, none exceeded), in the text and the regenerated table.

- **TOST margin.** Now justified by **deployment relevance** (a <5% difference does

not change instance-count/capacity planning), with the CV corroboration demoted.

- **Interaction test.** We report the interpretable effect (per-layer cross-engine

ratio ranges) alongside the blocked-permutation p, and state the statistic (randomized-block F on per-(layer,replicate) log-differences) and the within-replicate-block exchangeability assumption.

- **Primary vs secondary outcomes.** We added an **outcomes table** (Table 6)

separating the primary confirmatory outcomes (throughput, closed-loop p99) from secondary/exploratory controls.

- **Error/timeout attestation.** We state that HTTP errors, connection-acquisition

timeouts, and non-2xx responses were **zero across all 90 primary cells**; nonzero timeouts appear only in the deliberate open-loop overload runs.

Minor points (§7)

We adopted the wording changes: softened "production HTTP framework," "premiere," and "dominates perceived performance"; clarified the tuned baselines as a "practitioner-tuned reference / throughput ceiling" rather than a "lower bound"; reported the p99 sample size ($\approx$ throughput $\times$ 12 s completed requests, via autocannon's HdrHistogram, sub-millisecond, ms-rounded); clarified the knex/PostgreSQL/insert n=24 cell (the tabulated adjacent-pair tests are on the deep fetch, where every cell has the full 25 replicates); and de-duplicated recurring numbers flagged as repetitive. On the title we kept "for PostgreSQL and MySQL"; we are happy to add "at the HTTP level" if the editor prefers. On prior Prisma figures, we now state that absolute speedups are **not comparable given differing versions, hardware, workload, and harness**, rather than "not reproducible." Novelty claims (§10) are scoped to "in the sources we searched."

Artifact (§9)

The replication package now includes: explicit `LICENSE-code` (MIT) and `LICENSE-text` (CC-BY 4.0, extended to the datasets); `schema/db-config.md` documenting the effective server configuration and schema/index DDL for both engines; the raw per-cell and open-loop JSON (not only aggregated tables); an extended `MANIFEST.md` with a run-id $\rightarrow$ table-cell provenance note; refreshed `checksums.sha256`; and **unit tests for every statistical estimator** (`bench/stats.test.mjs`, 19 cases, `npm test`) against hand-computed values and closed-form properties. The generative-AI declaration now includes an **AI-assisted-code verification** paragraph (unit-tested estimators, cell-level provenance, and the byte-identical correctness cross-check).

Point-by-point answers to the reviewer's questions

1. **Adjacency / $\pm$5% margin fixed a priori?** Yes, chosen before analysis but not

preregistered; relabelled as planned contrasts reported descriptively. Repo history is available.

1. **Blocked-interaction statistic and exchangeability?** Randomized-block F on the

per-(layer,replicate) PostgreSQL-minus-MySQL log-throughput differences; layer labels are exchangeable within each replicate block under the no-interaction null.

1. **Is $\pm$5% CV-derived?** No — it is a deployment-relevance threshold

(instance-count/capacity planning); the CV is only corroborative.

1. **p99 sample size and instrument?** Each p99 is over $\approx$ throughput $\times$ 12 s

completed requests, from autocannon's HdrHistogram (sub-ms, reported ms-rounded).

1. **Errors/timeouts in the primary matrix?** Zero non-2xx, errors, and timeouts

across all 90 cells (fields confirmed and now stated).

1. **What does byte-equality compare?** The final serialized JSON response bodies

(`bench/verify.mjs`); status codes and health are checked; headers are not compared.

1. **Serialization determinism?** TZ=UTC, ISO-8601 dates, numeric coercion, BigInt→Number, canonical key order (`src/adapters/_canon.mjs`).

1. **Insert semantics across layers?** Single autocommit `INSERT ... RETURNING id` (pg) / `insertId` (mysql2) with identical return semantics; the transactional variant uses one transaction (`createThread`).

1. **What do the tuned baselines change?** They reuse prepared statements per pooled connection; otherwise identical.

1. **Cache/buffer state across cells?** Buffer pools are warm by design (memory-resident seed); randomized cell order and a fresh server boot per replicate mitigate carryover; the new post-restart check (S20) confirms the ranking survives a cold start.

1. **MySQL insert ceiling mechanism?** Now directly measured via performance_schema (S19): a shared redo-log/binlog commit flush.

1. **Prisma lead mechanism?** Observation (≈5× CPU, concurrency-dependent) plus the documented in-process engine; pipelining is labelled inference.

1. **ORM defaults (tracking/hooks/validation)?** Library defaults, disabled or request-scoped on the read path; documented per adapter (S16, METHODOLOGY.md).

1. **Were maintainers consulted?** No, deliberately, to preserve independence; disclosed as a limitation, with the artifact open for scrutiny.

1. **Was logging disabled?** Yes, disabled and verified at runtime on all eleven adapters (S16).

1. **What does the equal-CPU control pin?** The server process (`taskset`), with the database and load generator on disjoint cores.

1. **Does combined CPU include the generator / I/O wait?** The load generator is excluded; kernel and I/O-wait are not separately isolated, now disclosed.

1. **Are all table cells regenerable?** Yes, each from raw JSON via a committed script (`MANIFEST.md` maps every cell).

1. **AI-assistance verification?** Byte-identical correctness cross-check, unit-tested estimators, and independent hand-recomputation of key statistics.

1. **Effective word count?** 14,996 (`word-count.pdf`), under the 15,000 limit.

We believe the paper now claims exactly what its design supports, backed by four new experiments and a fully unit-tested, provenance-tracked artifact. We thank the reviewer for pushing it to this point.